National University of Singapore

CS1101S — Programming Methodology

AY2025/2026 Semester 2

Final Assessment

Time allowed: 2 hours

QUESTION PAPER

INSTRUCTIONS

1. This **QUESTION PAPER** contains **19 Questions** in **6 Sections**, and comprises **11 printed pages**, including this page.
2. You are also provided with an **ANSWER SHEET** to write your answers. It comprises **8 printed pages**.
3. Use a pen or pencil to **write** your **Student Number** in the designated space on the front page of the **ANSWER SHEET**, and **shade** the corresponding circle **completely** in the grid for each digit or letter. DO NOT WRITE YOUR NAME!
4. You must **submit only** the **ANSWER SHEET** and no other documents. Do not tear off any pages from the ANSWER SHEET.
5. All questions must be answered in the space provided in the **ANSWER SHEET**; no extra sheets will be accepted as answers.
6. Write legibly with a **pen** or **pencil** (do not use red color). Untidiness will be penalized.
7. For **multiple choice questions (MCQ)**, **shade** in the **circle** of the correct answer **completely**.
8. The full score of this assessment is **100** marks.
9. This is a **Closed-Book** assessment, but you are allowed to bring with you one double-sided **A4 / letter-sized sheet** of handwritten or printed **notes**.
10. Where programs are required, write them in the **Source §3** language. A **reference** of some of the **pre-declared functions** is given in the **Appendix** of the Question Paper.
11. In any question, unless it is specifically allowed, your answer **must not use functions** given in, or written by you for, other questions.

Section A: Pair Structures [16 marks]

For each of the following Source §3 programs, draw the *box-and-pointer diagram* for the values of the indicated names at the end of the program evaluation. **Clearly show where each name is pointing to. For pairs that are *identical* (===), only one box must be shown. Pairs that are *not identical* must be drawn as separate boxes.**

(A1) [4 marks]

Draw the box-and-pointer diagram for the value of **xx**.

```
let xx = list(4, 5, 6);
set_head(xx, tail(xx));
set_tail(xx, tail(head(xx)));
```

(A2) [4 marks]

Draw the box-and-pointer diagram for the values of **xx** and **yy**. (Refer to the Appendix for the implementation of `map`.)

```
const xx = list(list(2), 3);
const yy = map(x => x, xx);
set_head(head(yy), 8);
```

(A3) [4 marks]

Draw the box-and-pointer diagram for the values of **xx** and **yy**. (Refer to the Appendix for the implementation of `append`.)

```
let xx = list(2, list(3));
let yy = append(xx, xx);
```

(A4) [4 marks]

Draw the box-and-pointer diagram for the value of **xx**. (Refer to the Appendix for the implementation of `accumulate`.)

```
let xx = accumulate((x, y) => list(y, x),
                    list(8), list(2, 3, 4));
```

Section B: Orders of Growth [12 marks]

For each question in this section, choose the order of growth that best characterize the running time of the function `fun`. Note that N is a non-negative number value.

(B1) [3 marks] (MCQ)

```
function fun(N) {
    let acc = 0;
    for (let i = 1; i <= N / 2; i = i + 1) {
        for (let k = 1; k <= i; k = k + 1) { acc = acc + 1; }
    }
    return acc;
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

(B2) [3 marks] (MCQ)

```
function fun(N) {
    let acc = 0;
    for (let i = 1; i <= N; i = i + 1) {
        for (let k = 1; k <= 1000000; k = k + 1) { acc = acc + 1; }
    }
    return acc;
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

(B3) [3 marks] (MCQ)

```
function fun(N) {
    let acc = 0;
    for (let d = 2; d <= 1000; d = d + 1) {
        for (let j = N; j >= 1; j = j / d) { acc = acc + 1; }
    }
    return acc;
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

(B4) [3 marks] (MCQ)

```
function fun(N) {
    let acc = 0;
    for (let i = 1; i <= N; i = i + 1) {
        for (let j = N; j >= 1; j = j / 2) { acc = acc + 1; }
        for (let k = 1; k <= N; k = k * 2) { acc = acc + 1; }
    }
    return acc;
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

Section C: Program Evaluation [12 marks]

We assume that in **Source §3**,

(1) the left operand of a binary operation is evaluated before the right operand, and

(2) when there are multiple arguments in a function application, the argument expressions are evaluated from left to right, i.e. the leftmost argument expression is evaluated first, and the rightmost last.

For each of the programs in this section, **write the sequence of values printed** by the **`display`** function when the program is evaluated. The `display` function displays/prints the value of its input argument in the REPL, *and* returns the value of its input argument. For example, `display(2 * 5)` prints `10` and returns `10`; and `display(pair(1, 2))` prints `[1, 2]` in box notation and returns the pair `[1, 2]`.

(C1) [4 marks]

```
const D = display;

function fun(x, y) {
    return D( D(y * 2) + D(3 * x) ); // (y * 2) + (3 * x)
}

D( fun( D(4), D(5) ) ); // equivalent to fun(4, 5);
```

(C2) [4 marks]

```
const D = (m, x) => { display(m); return x; };

display("A");
const foo = D( "fn", (x, y) => D("+", D("x", x) + D("y", y)) );
display("B");
display( foo( D(12, 12), D(9, 9) ) );
```

(C3) [4 marks]

```
function foo(n) {
    display(n);

    if (n <= 4) {
        return n;
    } else {
        const result = foo(n - 3) + foo(n - 1);
        display(result, "r:");
        return result;
    }
}
foo(6);
```

Section D: Environment Model [10 marks]

(D1) [10 marks]

For the following Source program, draw the diagram to show the environment during the evaluation of the program. Show all the frames that are created during the program evaluation (except the global environment frame), and show the final value of each binding.

```
function foo(X) {
    const Y = 2 * X;

    function gee(Z) {
        const W = Y + Z;
        return KK + 3 * W;
    }

    return gee;
}

let KK = 10;
const A = foo(5);
const AA = A(6);

KK = 15;
const B = foo(7);
const BB = B(8);
```

Section E: Big-Integers [16 marks]

A ***big-integer*** is a representation of a **non-negative integer number** using a **list** whose elements are the decimal digits of the number, and the least significant digit is the first element of the list. For example, the decimal number `20536` is represented as the big-integer `list(6, 3, 5, 0, 2)`.

The integer `0` (zero) is represented as `list(0)`, but **leading zeros are not allowed** in all other big-integers. For example, the integer `913` cannot be represented as `list(3, 1, 9, 0)`.

Note that big-integers can represent integers with hundreds and thousands of digits, which is beyond what Source numbers can represent.

(E1) [8 marks]

Complete the declaration of the function `num_value`, which takes as argument a big-integer and returns the represented number value, which we assume is within the range that can be accurately represented as a Source number. For example,

```
num_value(list(6, 3, 5, 0, 2));  // returns 20536
num_value(list(1));  // returns 1
```

The running time of your function must have an order of growth of $O(n)$, where n is the length of the input list. The input list must not be modified.

Write your answer only in the dashed-line box(es).

(E2) [8 marks]

Complete the declaration of the function `add_big_int`, which takes as arguments two big-integers and returns a big-integer that represents the sum of the input big-integers. For example,

```
const A = list(5, 0, 6);     // represents 605
const B = list(5, 8, 7, 9);  // represents 9785
add_big_int(A, B);  // returns list(0, 9, 3, 0, 1)
                    // Equivalent to 605 + 9785 = 10390
```

Note that the input big-integers could be representing numbers that are much larger than the range of integer values that can be accurately represented as Source numbers.

The running time of your function must have an order of growth of $O(n)$, where n is the total length of the input lists. The input lists must not be modified.

Write your answer only in the dashed-line box(es).

Section F: Arraytrees [34 marks]

An ***arraytree*** is either a **number** or a non-empty array whose elements are arraytrees.

Here are some example arraytrees:

- `75`
- `[32]`
- `[ 9, [ 3, 8, [6, 3] ], 5, [ [7] ] ]`

(F1) [8 marks]

Complete the declaration of the function `count_num`, which takes as argument an arraytree `T` and returns the number of numbers that are in `T`. For example,

```
count_num(75);    // returns 1
count_num([32]);  // returns 1
count_num([ 9, [ 3, 8, [6, 3] ], 5, [ [4] ] ]);  // returns 7
```

The input arraytree must not be modified.

Write your answer only in the dashed-line box(es).

(F2) [8 marks]

Complete the declaration of the function `rev_tree`, which takes as argument an arraytree `T` and returns a **new copy** of `T` but with all the arrays in `T` **reversed**. For example,

```
rev_tree(75);            // returns 75

rev_tree([32, 67, 45]);  // returns [45, 67, 32]

rev_tree([19, [23, 38, [46, 53]], 65, [[77]]]);
// returns [[[77]], 65, [[53, 46], 38, 23], 19]
```

The input arraytree must not be modified.

Write your answer only in the dashed-line box(es).

(F3) [4 marks]

Complete the declaration of the function `nest`, which takes as arguments a number `x` and a non-negative integer `n`, and returns a **`n`-level nested array** containing `x`. For example,

```
nest(58, 0);  // returns 58
nest(58, 1);  // returns [58]
nest(58, 2);  // returns [[58]]
nest(58, 3);  // returns [[[58]]]
```

(F4) [6 marks]

Complete the declaration of the function `nest_many`, which takes as argument a list of numbers, `L`, and returns an array `A`, where `A[i]` is an **`i`-level nested array** containing the element of `L` at position `i`. For example,

```
nest_many(list(3, 9, 4, 6));  // returns [ 3, [9], [[4]], [[[6]]] ]
nest_many(list());  // returns []
```

Your function may use the `nest` function from the preceding question.

The input list must not be modified.

(F5) [8 marks]

Complete the declaration of the function `replace_num`, which takes as arguments an arraytree `T`, and an array of numbers, `A`, and replaces all the numbers in `T` with the elements of `A`, in the order as demonstrated in the following example:

```
const T = [[0, 0], 0, [[0, [0]], 0, [[0]]]];
const A = [2, 1, 6, 7, 3, 5, 4];
replace_num(T, A);
T;  // value: [[2, 1], 6, [[7, [3]], 5, [[4]]]]
```

We assume that the number of numbers in the input arraytree `T` is equal to the length of the input array `A`. Note that the function does not return a value since the result is the modified `T`.

Write your answer only in the dashed-line box(es).

END OF QUESTIONS

Appendix

We assume the following pre-declared functions in Source §3 are declared as follows:

```
function map(f, xs) {
    return is_null(xs)
           ? xs
           : pair(f(head(xs)), map(f, tail(xs)));
}

function filter(pred, xs) {
    return is_null(xs)
           ? null
           : pred(head(xs))
           ? pair(head(xs), filter(pred, tail(xs)))
           : filter(pred, tail(xs));
}

function accumulate(f, initial, xs) {
    return is_null(xs)
           ? initial
           : f(head(xs), accumulate(f, initial, tail(xs)));
}

function append(xs, ys) {
    return is_null(xs)
           ? ys
           : pair(head(xs), append(tail(xs), ys));
}
```

The following **LISTS** functions are supported in Source §3:

- `pair(x, y)`: Makes a pair from `x` and `y`.
- `is_pair(x)`: Returns `true` if `x` is a pair and `false` otherwise.
- `head(x)`: Returns the head (first component) of the pair `x`.
- `tail(x)`: Returns the tail (second component) of the pair `x`.
- `set_head(p, x)`: Changes the pair `p` such that its head is `x`.
- `set_tail(p, x)`: Changes the pair `p` such that its tail is `x`.
- `is_null(xs)`: Returns `true` if `xs` is the empty list, and `false` otherwise.
- `is_list(x)`: Returns `true` if `x` is a list as defined in the lectures, and `false` otherwise. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of the chain of `tail` operations that can be applied to `x`.
- `list(x1, x2,..., xn)`: Returns a list with `n` elements. The first element is `x1`, the second `x2`, etc. Iterative process; time: $O(n)$, space: $O(n)$, since the constructed list data structure consists of n pairs, each of which takes up a constant amount of space.
- `length(xs)`: Returns the length of the list `xs`. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of `xs`.
- `map(f, xs)`: Returns a list that results from list `xs` by element-wise application of `f`. **Recursive process**; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `build_list(f, n)`: Makes a list with `n` elements by applying the unary function `f` to the numbers 0 to `n - 1`. Iterative process; time: $O(n)$, space: $O(n)$.

- `for_each(f, xs)`: Applies `f` to every element of the list `xs`, and then returns `true`. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of `xs`.
- `reverse(xs)`: Returns list `xs` in reverse order. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`. The process is iterative, but consumes space $O(n)$ because of the result list.
- `append(xs, ys)`: Returns a list that results from appending the list `ys` to the list `xs`. **Recursive process**; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `member(x, xs)`: Returns first postfix sublist whose head is identical to `x` (`===`); returns `null` if the element does not occur in the list. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of `xs`.
- `remove(x, xs)`: Returns a list that results from `xs` by removing the first item from `xs` that is identical (`===`) to `x`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `remove_all(x, xs)`: Returns a list that results from `xs` by removing all items from `xs` that are identical (`===`) to `x`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `filter(pred, xs)`: Returns a list that contains only those elements for which the one argument function `pred` returns `true`. **Recursive process**; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `enum_list(start, end)`: Returns a list that enumerates numbers starting from `start` using a step size of 1, until the number exceeds (`>`) `end`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`. For example, `enum_list(2, 5)` returns the list `list(2, 3, 4, 5)`.
- `list_ref(xs, n)`: Returns the element of list `xs` at position `n`, where the first element has index 0. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of `xs`.
- `accumulate(op, initial, xs)`: Applies binary function `op` to the elements of `xs` from right-to-left order, first applying `op` to the last element and the value `initial`, resulting in r_1, then to the second-last element and r_1, resulting in r_2, etc, and finally to the first element and r_{n-1}, where n is the length of the list. Thus, `accumulate(op, zero, list(1,2,3))` results in `op(1, op(2, op(3, zero)))`. **Recursive process**; time: $O(n)$, space: $O(n)$, where n is the length of `xs`, assuming `op` takes constant time.

Some other functions supported in Source §3:

- `is_boolean(x)`: Returns `true` if `x` is a boolean value, and `false` otherwise.
- `is_number(x)`: Returns `true` if `x` is a number, and `false` otherwise.
- `is_string(x)`: Returns `true` if `x` is a string, and `false` otherwise.
- `is_array(x)`: Returns `true` if `x` is a boolean value, and `false` otherwise.
- `array_length(x)`: Returns the current length of array `x`.
- `display(v)`: Displays the value `v` in the REPL and returns the value `v`.
- `stringify(v)`: Returns a string that represents the value `v`. For example, `stringify(123)` returns the string `"123"`, and `stringify(false)` returns the string `"false"`.
- `math_floor(x)`: Returns the largest integer that is equal to or less than the number `x`.
- `math_ceil(x)`: Returns the smallest integer that is equal to or greater than the number `x`.
- `math_sqrt(x)`: Returns the square root of the number `x`.
- `math_pow(base, exp)`: Returns the result of raising `base` to the power of `exp`.
- `math_min(x1, x2,..., xn)`: Returns the smallest of the `n` arguments. Time: $O(n)$, space: $O(1)$.
- `math_max(x1, x2,..., xn)`: Returns the largest of the `n` arguments. Time: $O(n)$, space: $O(1)$.

——— END OF PAPER ———

National University of Singapore

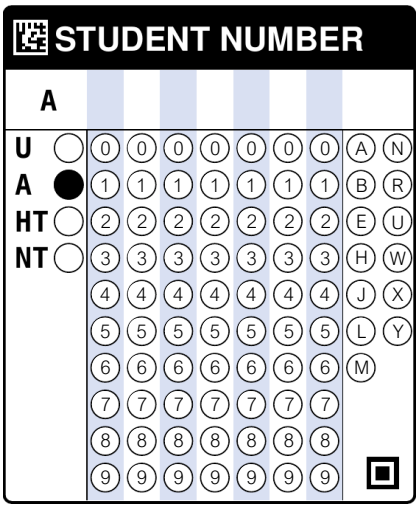
CS1101S — Programming Methodology

AY2025/2026 Semester 2

Final Assessment

Time allowed: 2 hours

ANSWER SHEET



(A1) [4 marks]

(A2) [4 marks]

(A3) [4 marks]

(A4) [4 marks]

(B1) [3 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(B2) [3 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(B3) [3 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(B4) [3 marks] ○ **A** ○ **B** ○ **C** ○ **D** ○ **E** ○ **F** ○ **G** ○ **H**

(C1) [4 marks]

(C2) [4 marks]

(C3) [4 marks]

(D1) [10 marks]

(E1) [8 marks]

```
function num_value(B) {
    return [                                                    ]
           [                                                    ]
         ? [                                                    ]
           [                                                    ]
         : [                                                    ]
           [                                                    ]
           [                                                    ]
}
```

(E2) [8 marks]

```
function add_big_int(A, B) {
    function add(x, y, carry) {
        if (is_null(x) && is_null(y)) {
            return carry === 0 ? null : list(carry);
        } else {
            const xdigit = is_null(x) ? 0 : head(x);
            const ydigit = is_null(y) ? 0 : head(y);

            const sum = [                                       ]
                        [                                       ]

            const new_digit = [                                 ]
                              [                                 ]

            const new_carry = [                                 ]
                              [                                 ]

            return pair(new_digit,
                        [                                       ]
                        [                                       ]
                        [                                       ]
                        [                                       ]
                        [                                       ]
                        ); // pair
        }
    }
    return add(A, B, 0);
}
```

(F1) [8 marks]

```
function count_num(T) {
    if (is_number(T)) {
        [                                                    ]
    } else {
        let count = 0;
        [                                                    ]
        [                                                    ]
        [                                                    ]
        [                                                    ]
        [                                                    ]
        [                                                    ]
        [                                                    ]
        return count;
    }
}
```

(F2) [8 marks]

```
function rev_tree(T) {
    if (is_number(T)) {
        [                                                    ]
    } else {
        const B = [];
        [                                                    ]
        [                                                    ]
        [                                                    ]
        [                                                    ]
        [                                                    ]
        [                                                    ]
        [                                                    ]
        return B;
    }
}
```

(F3) [4 marks]

```
function nest(x, n) {




}
```

(F4) [6 marks]

```
function nest_many(L) {




}
```

(F5) [8 marks]

```
function replace_num(T, A) {
    let next_idx = 0;

    function replace(t) {
        const len = array_length(t);
        for (let i = 0; i < len; i = i + 1) {
            
        } // for
    }
    replace(T);
}
```

——— END OF ANSWER SHEET ———